

Grokking 现象的复现与影响因素探究：模加法任务实验报告

毛志远 2300012108

蔡琳珊 2400010722

陈佳鸿 2400010751

Abstract

本文复现了 *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets and beyond* 中模加法任务的实验，聚焦数据比例 α 、网络架构、优化器/正则化、任务复杂度（求和项数量 K ）四类核心因素，系统探究其对 grokking 现象（过拟合后延迟泛化）的调控作用。实验结果表明，数据比例越高，grokking 的发生效率越高；Gated MLP、1D-CNN 等架构适配模加法任务的 grokking 过程，而 SIREN 架构完全无法实现泛化；权重衰减是触发 grokking 的必要条件，不同优化器仅影响 grokking 的效率；任务复杂度的提升会直接导致 grokking 现象消失，仅低 K 值的简单模加法任务能观测到该现象。

1 引言

Grokking 现象是小算法数据集领域中观测到的特殊泛化规律，具体表现为模型在训练阶段先完全过拟合训练数据（训练准确率达到饱和），但测试准确率长期处于低水平，随后在持续训练中测试准确率突然延迟上升至完美泛化状态。该现象首次在模运算任务中被发现，为理解模型从“记忆数据”到“学习规律”的转变机制提供了重要视角。本实验以模加法任务 $(x_1, \dots, x_K) \mapsto \left(\sum_{k=1}^K x_k\right) \bmod p$ 为核心载体，基于原论文的实验框架完成四项子任务探究，通过控制变量法逐一分析数据比例、网络架构、优化器/正则化、任务复杂度对 grokking 现象的影响，明确各因素的作用机制与调控规律。

2 实验设置

本实验全程基于 PyTorch 深度学习框架实现，为保证实验可复现性与计算效率，设定了统一的核心实验配置。实验的核心任务为模加法运算，为避免模型预先学习到“整数”的固有概念，所有输入整数均被编码为 one-hot 向量形式参与训练；实验的核心变量分为三类，分别是数据比例 α （训练数据量与总数据量 p^2 的比值）、任务复杂度（模加法运算中的求和项数量 K ），以及作为调控因素的网络架构、优化器类型与正则化策略（主要为权重衰减）；此外，为降低计算成本且不影响实验结论的普适性，本实验选用较小的质数 p 完成所有模加法运算，确保在有限计算资源下仍能观测到完整的 grokking 过程。详细的实验设置参见附录部分A。

3 实验结果分析

3.1 Subtask 1: 数据比例 α 对 grokking 的影响

Subtask 1 以数据比例 α 为唯一变量，通过控制不同的训练数据占比开展对比实验，探究其与 grokking 过程的关联规律。在 $\alpha = 0.5$ 的实验组中（图 1），实验观测到了典型的 grokking 过程：模型的训练准确率在约 10^3 步训练后迅速饱和至 100%，呈现出明显的过拟合特征，但测试准确率在此阶段仍处于低水平；随着训练步数的增加，测试准确率开始延迟上升，最终接近 100% 的完美泛化状态。图 2 展示了数据效率相关的实验结果，通过对比不同 α 值下模型达到 99%，测试准确率所需的训练步数，发现 α 值越小（即训练数据量越少），模型完成泛化所需的步数越多，而随着 α 值增大，所需步数呈快速下降趋势，这一结果直接证明了更高的训练数据比例能够显著提升 grokking 的发生效率。为排除原始准确率曲线波动对实验结论的干扰，本实验还对曲线进行了平滑处理（图 3），平滑后的曲线清晰呈现出“训练准确率先饱和、测试准确率后追平”的核心特征，进一步验证了 grokking 现象的本质是模型从记忆数据到学习规律的转变，而非曲线波动导致的偶然结果。

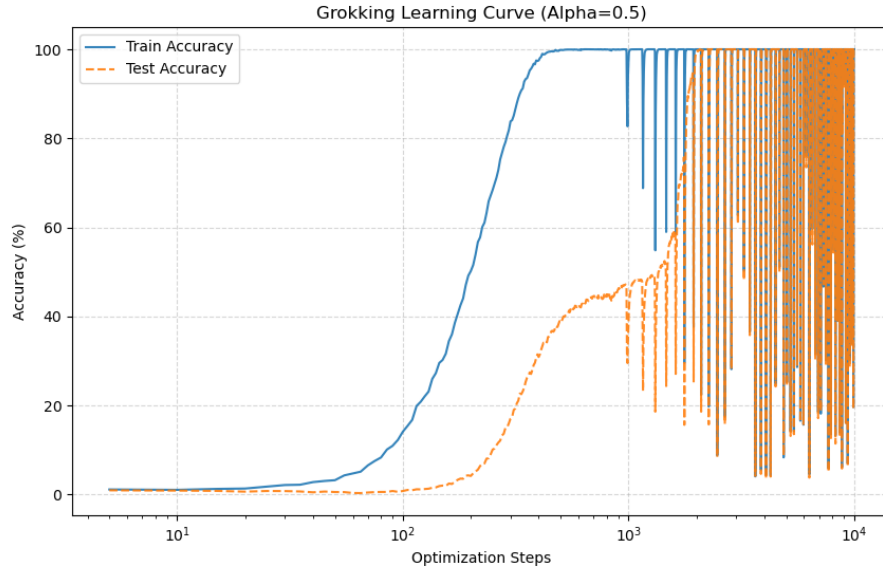


图 1: $\alpha = 0.5$ 的 grokking 曲线

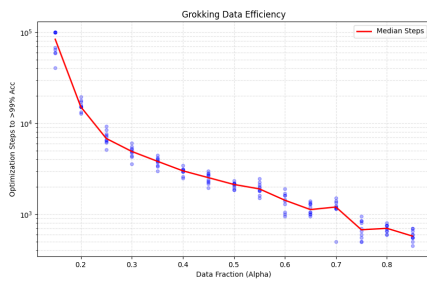


图 2: 不同 α 值下观察到 grokking 所需的优化步数

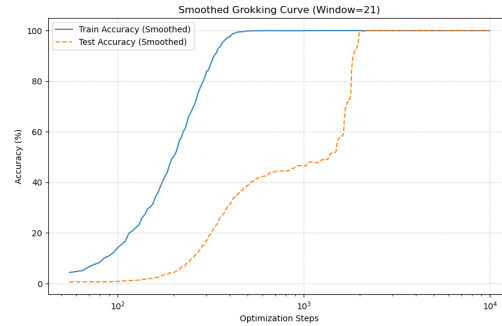


图 3: 平滑处理的 grokking 曲线

3.2 Subtask 2: 网络架构对 grokking 的影响

Subtask 2 聚焦网络架构的适配性问题，通过构建多种不同类型的网络架构开展平行实验，对比各架构下 grokking 现象的发生情况，实验结果对应图（4）。实验中选取了 Gated MLP、1D-CNN、LSTM、Deep MLP、SIREN (Periodic)、Simple MLP、RetNet MLP 等多种典型架构，其中 Gated MLP、1D-CNN、LSTM 与 Deep MLP 均展现出典型的 grokking 特征：训练过程中训练准确率先快速饱和至 100%，随后测试准确率低延迟上升并最终追平训练准确率；而 SIREN (Periodic) 架构的表现则完全不同，其训练准确率虽能逐步上升，但测试准确率始终维持在低迷水平，完全无法实现泛化；Simple MLP 与 RetNet MLP 架构虽能最终接近完美泛化准确率，但泛化过程中的准确率曲线存在明显波动，稳定性弱于前四类架构。上述结果充分说明，网络架构的适配性是 grokking 现象发生的前提条件，只有与模加法这类算法任务特性相匹配的架构，才能触发从记忆到泛化的转变，而 SIREN、Vanilla RNN 架构因结构特性无法捕捉模加法的运算规律，因此无法实现 grokking。

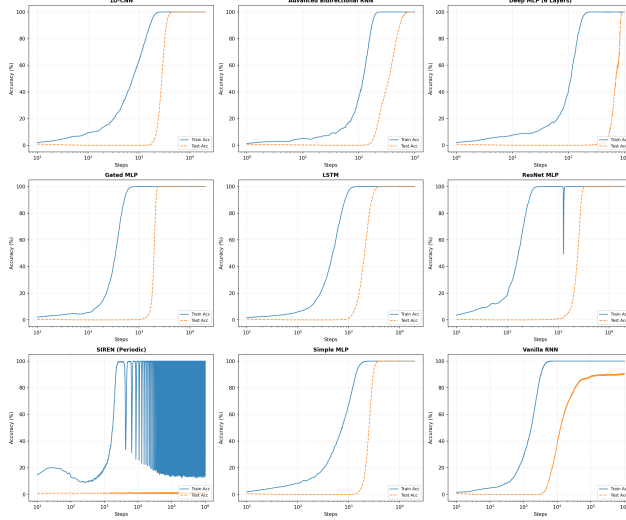


图 4: 不同网络架构的 Grokking 现象对比

3.3 Subtask 3: 优化器与正则化对 grokking 的影响

Subtask 3 以优化器类型与正则化策略为调控变量，探究其对 grokking 现象的影响机制，实验结果对应图 5 与图 6。实验的核心对比维度为权重衰减的有无以及不同优化器的效率差异，在以 AdamW(WD=1) 为基线的实验组中 (WD=1 表示权重衰减系数为 1)，当 $\alpha > 0.3$ 时，模型的测试准确率出现明显的“跳变”特征，直接上升至 100%，呈现出典型的 grokking 现象；而在无权重衰减的 Adam (No WD) 实验组中，模型的测试准确率仅随 α 值的增大缓慢上升，全程未观测到 grokking 现象，这一结果证明权重衰减是触发 grokking 的必要条件。同时，超参数调整实验表明，高权重衰减、低学习率等配置仅会改变 grokking 的触发阈值。此外，不同优化器的效率对比结果显示，AdamW(Low LR) (低学习率版本) 完成 grokking 所需的训练步数显著多于基线 AdamW；Full-Batch GD (全批量梯度下降) 的泛化过程则存在剧烈的准确率波动，甚至出现准确率回落的情况；SGD 与 RMSprop 优化器在 α 值足够大时虽能触发 grokking，但整体效率低于基线 AdamW 组。

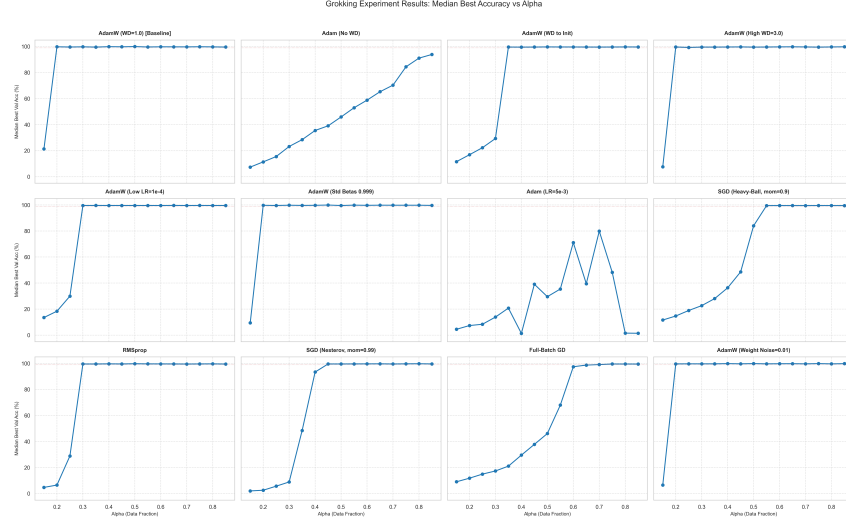


图 5: 不同优化器与正则化策略下的 α -best accuracy 曲线

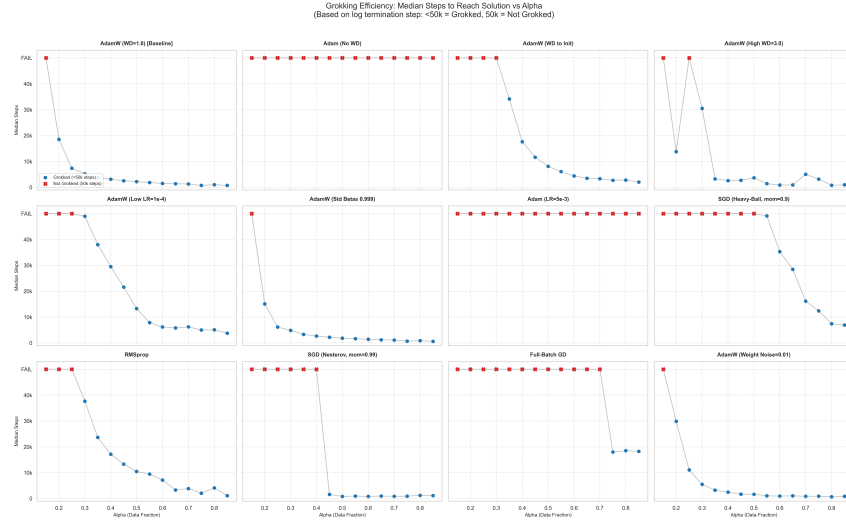


图 6: 不同优化器与正则化策略下发生观察到 grokking 所需的优化步数（取 5 个随机种子的中位值）

3.4 Subtask 4: 任务复杂度对 grokking 的影响

Subtask 4 将任务复杂度（以求和项数量 K 为量化指标）作为核心变量，分析其对 grokking 现象的调控作用，实验结果对应图 1。实验中设置了 $K = 2\ 3\ 4\ 5$ 四组不同复杂度的模加法任务，同时由于输入数据具有显著的对称性特征，我们在部分实验组进行了 PE+Mask (positional encoding+causal mask) 消融实验。在 $K = 2$ 与 $K = 3$ 的低复杂度实验组中，PE+Mask 组合配置下能清晰观测到 grokking 现象：训练准确率先快速饱和，测试准确率在延迟后上升至接近 100%；而当 $K = 4$ 与 $K = 5$ 时，无论是否采用 PE+Mask 配置，模型的训练准确率与测试准确率均长期维持在低水平，grokking 现象完全消失。这一结果表明，任务复杂度的提升会显著增加模型泛化的难度，grokking 现象仅能在低 K 值的简单模加法任务中发生，当求和项数量超出一定阈值后，模型无法从记忆数据转向学习运算规律，因此无法实现延迟泛化。

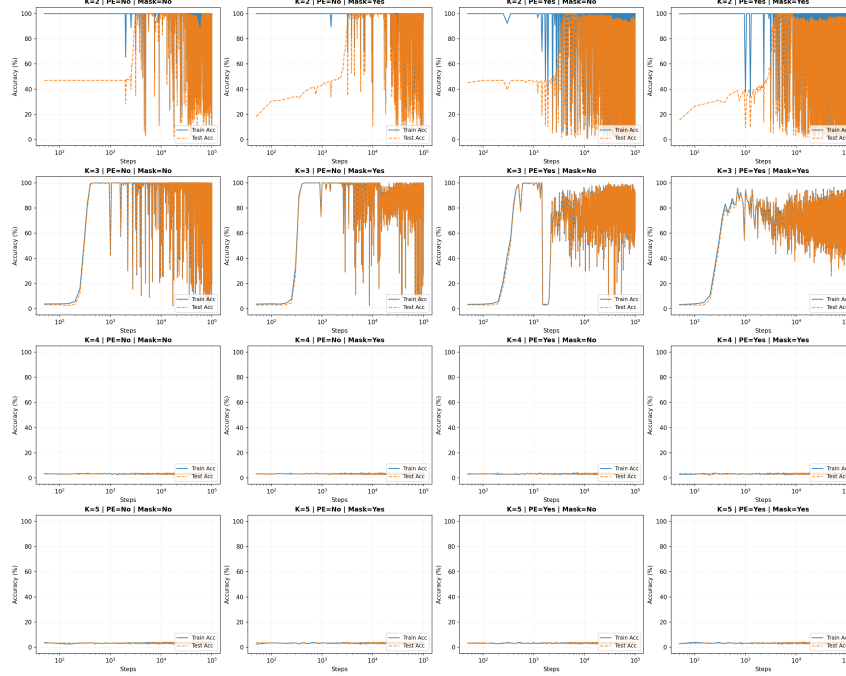


图 7: 不同任务复杂度（以求和项数量 K 为量化指标）下的 Grokking 现象对比

3.5 Subtask 5: grokking 现象的解释

结合所有子任务的实验结果，可对 grokking 现象的本质与发生逻辑形成完整解释：grokking 的核心是模型在训练过程中从“记忆训练数据”向“学习任务底层规律”的阶段性转变，这一转变的发生需要满足三项前提条件，即任务本身为低复杂度的算法任务（如低 K 值的模加法）、所选用的网络架构与任务特性相适配、训练过程中存在权重衰减等正则化约束。从过程机制来看，训练初期模型通过记忆训练数据的方式拟合标签，因此呈现出过拟合特征；而权重衰减的正则化约束会限制模型的记忆能力，迫使模型放弃单纯的记忆策略，转而学习模加法运算的通用逻辑，最终实现泛化；从各因素的影响逻辑来看，数据比例决定了这一转变的效率，网络架构与优化器/正则化决定了转变的可能性，而任务复杂度则决定了转变的可行性，只有在所有条件均满足时，grokking 现象才会发生。

3.5.1 记忆阶段与结构阶段

实验中普遍观测到，模型在训练初期即可在有限训练数据上迅速达到接近 100% 的训练准确率，但此时测试准确率仍长期维持在接近随机猜测的低水平。这表明，在该阶段模型主要依赖高复杂度的记忆型表示对训练样本进行逐点拟合，而尚未学习到模加法任务所蕴含的通用计算规律。

随着训练的持续进行，模型参数不断更新，但训练准确率几乎不再发生变化，验证准确率却在某一时间点突然跃升至接近 100%。这一“延迟跃迁”表明，模型在训练后期完成了从局部记忆到全局结构的转变，即成功内化了模加法这一算法任务的底层规律。

由此可以将 grokking 过程概括为两个阶段：

- 记忆阶段：模型以高复杂度参数配置记忆训练样本
- 结构阶段：模型转向学习可泛化的算法表示，从而实现整体泛化

3.5.2 损失景观视角下的 grokking 过程分析

为进一步理解 grokking 现象的内在机制，本实验根据 Subtask 2 的实验数据通过 PCA 降维绘制了 simpleMLP 与 Transformer 模型的损失景观及训练过程的优化路径并标出 grokking 发生的节点（图 8），解析模型从“记忆”到“泛化”的转变过程。

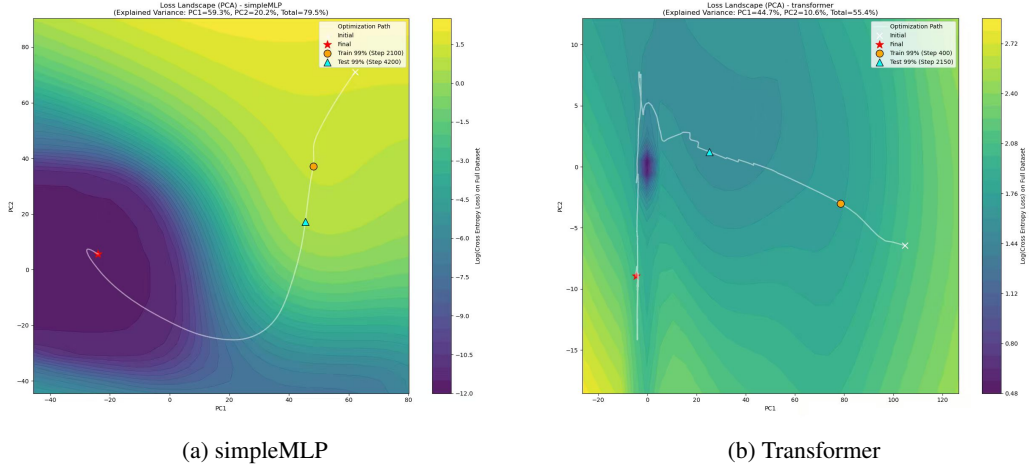


图 8: 模型的损失景观及训练过程的优化路径

对比图 8 可以看出，simpleMLP 的损失景观是“单核心连续低损失区”，泛化区域集中且无高损失壁垒，优化路径平滑顺畅从过拟合点到泛化点沿连续低损失带直接移动，无明显波动；Transformer 的损失景观是“分散式低损失区”，泛化区域与过拟合点之间被高损失区域分隔路径曲折反复，参数需突破多重高损失壁垒才能抵达泛化区，过程中存在多次“试错式移动”。尽管有诸多不同，两张图的优化路径均可分为三个阶段：

- 第一阶段：模型初始点均位于高损失区域，参数随机分布导致无法捕捉任务规律
- 第二阶段：训练早期快速收敛到过拟合状态（即训练准确率达到 99%，simpleMLP 为 Step 2400，Transformer 为 Step 350），此时参数停留在“训练数据适配的局部低损失点”，测试准确率仍低迷
- 第三阶段：持续训练后，损失继续下降，参数均从过拟合点移动到泛化点（即测试准确率达到 99%，simpleMLP 为 Step 5150，Transformer 为 Step 2050），完成从记忆到泛化的跳转

故而，损失景观的“连通性”与“集中性”决定 grokking 效率——连续集中的低损失区能缩短优化路径，让泛化更快发生；而分散隔离的低损失区会延长路径、增加波动，但不影响 grokking 的最终发生。无论损失景观结构如何，当损失景观中存在独立的泛化低损失区，且过拟合点与泛化点之间存在可通行的优化路径，grokking 就能发生，模型就能在持续训练中突破过拟合，实现延迟泛化。由此，我们可以根据之前的实验结果解释 grokking 发生的条件和原因。

3.5.3 数据比例 α 处于合理区间

数据比例需为“部分训练、部分验证”的场景（通常在 0.2-0.8 之间）：过小（如 <0.2 ）时训练数据不足，泛化信号微弱，损失景观中泛化低损失区模糊，过拟合点与泛化点之间路径断裂；过大（如 $=1.0$ ，全量数据训练）时无验证集可泛化，损失景观中仅存在全数据适配的单

一低损失点，不存在从过拟合到泛化的路径跳转；而合理 区间内，训练数据足够提供任务规律信号，验证集可引导泛化低损失区形成，过拟合点与泛化点之间的路径因规律信号支撑而连通。

3.5.4 权重衰减（隐式正则化）的关键作用

Subtask 3 的实验结果清晰表明，权重衰减是触发 *grokking* 的必要条件。实验中通过 AdamW 优化器设置权重衰减（WD=1），其本质是对模型参数施加 L2 约束——这种约束会惩罚过度复杂的记忆型参数，重构损失景观结构：消除“孤立的虚假局部低损失点”（仅适配训练数据记忆的参数组合），形成“连续的泛化低损失区”（适配任务规律的参数组合，更具泛化能力的算法解通常对应范数更小、结构更简单的表示方式），让参数在持续训练中可逐步探索至泛化区。反之，无权重衰减时（如 Adam 优化器 No WD），损失景观呈现“离散孤岛式低损失区”，过拟合点与泛化点之间无连通路程，*grokking* 无法发生。

这一解释也自然说明了不同优化器与超参数设置的影响规律：优化器类型与学习率主要决定模型到达该结构解所需的时间尺度，而权重衰减则决定这一结构解是否在优化路径中“可达”。

3.5.5 网络架构与任务特性匹配

网络架构与任务复杂度的实验结果进一步表明，*grokking* 并非对所有模型与任务普遍成立。仅当任务本身具有清晰、低复杂度的算法结构（如低 K 值的模加法），且模型架构具备与之匹配的归纳偏置时，模型才可能在正则化约束下完成从记忆到结构的跃迁。

例如，Gated MLP、1D-CNN 等架构在实验中能够稳定触发 *grokking*，而 SIREN 架构则完全无法实现泛化，表明其参数化方式不利于表达模加法所需的离散运算结构；同时，当任务复杂度提升至 $K \geq 4$ 时，即便引入正则化与输入增强手段，模型也难以学习到统一的计算规律，*grokking* 现象随之消失。这说明，*grokking* 的发生不仅依赖训练过程，还受到任务可学习性本身的严格限制。

4 结论

本实验通过控制变量法系统探究了数据比例 α 、网络架构、优化器/正则化、任务复杂度 K 四类因素对模加法任务中 *grokking* 现象的影响。研究发现，数据比例 α 与 *grokking* 效率呈正相关，更高的训练数据占比能让模型更快完成从记忆到泛化的转变；网络架构的适配性是 *grokking* 发生的前提，仅 Gated MLP、1D-CNN 等与算法任务匹配的架构能触发该现象；权重衰减是 *grokking* 的必要条件，不同优化器仅影响 *grokking* 的发生效率，而无权重衰减时 *grokking* 完全无法发生；任务复杂度 K 的提升会直接导致 *grokking* 消失，仅低复杂度的模加法任务能观测到该现象。本实验完整复现了 *grokking* 现象并明确了各核心因素的调控规律，为理解小算法数据集的泛化机制提供了实验支撑，也为后续优化 *grokking* 过程、拓展适用任务场景提供了参考方向。

A 附录

A.1 数据集生成

数学定义

本研究的核心任务为**模加法运算**，数学定义为：

$$(x_1, x_2, \dots, x_K) \mapsto \left(\sum_{k=1}^K x_k \right) \bmod p$$

其中：

- p 为质数（子任务 1-3 中取 $p = 97$ ，子任务 4 中为减小计算量取 $p = 31$ ）
- $x_k \in \mathbb{Z}_p = \{0, 1, \dots, p-1\}$
- 主要研究 $K = 2$ 的二元模加法（子任务 1-3），扩展研究 $K \in \{2, 3, 4, 5\}$ 的多变量模加法（子任务 4）

数据划分与比例

- **总数据量**： $N_{\text{total}} = p^K$ （所有可能的输入组合）
- **训练集比例**： $\alpha = \frac{N_{\text{train}}}{N_{\text{total}}}$ ，取值范围为 $[0.15, 0.85]$
- **划分方式**： 随机打乱所有可能方程后按比例划分训练集和测试集
- **固定随机种子**： 确保实验结果的可复现性

输入编码方式

为实现模型的符号学习特性（而非利用数字的语义信息），采用以下编码方案：

1. **符号化表示**： 每个整数 $0 \leq x < p$ 视为独立符号
2. **操作符编码**：
 - 加号"+": 编码为 token p
 - 等号"=": 编码为 token $p + 1$
3. **序列格式**：
 - 二元运算： $[x_1, +, x_2, =] \rightarrow$ 预测结果
 - K 元运算： $[x_1, +, x_2, +, \dots, x_K, =] \rightarrow$ 预测结果
4. **嵌入层**： 使用 `nn.Embedding` 将离散符号映射为连续向量，维度 $d_{\text{model}} = 128$

A.2 模型架构

Transformer（子任务 1）

本研究采用 **Decoder-only Transformer** 架构，严格复现了原始论文配置：

- **嵌入维度**： $d_{\text{model}} = 128$
- **层数**： $n_{\text{layers}} = 2$
- **注意力头数**： $n_{\text{heads}} = 4$

- **前馈网络维度**: $d_{ff} = 4 \times d_{model} = 512$
- **位置编码**: 正弦余弦位置编码 (Sinusoidal Positional Encoding)
- **注意力掩码**: 因果掩码 (Causal Mask), 防止信息泄漏
- **输出层**: 线性层将隐藏状态映射到词汇表 (大小 $p + 3$)

多样化的网络架构 (子任务 2)

为探究 Grokking 现象的普适性, 实现了以下 8 种网络架构:

表 1: Subtask 2 网络架构参数配置

模型类型	关键特性	隐藏维度	层数	特殊配置
Simple MLP	两层全连接 + ReLU	128	2	-
Gated MLP (GLU)	门控线性单元增强非线性	128	2	GLU 门控
SIREN MLP	周期性激活函数	128	2	$\text{Sine}(\omega_0 = 30)$
1D-CNN	卷积处理序列	128	2	$\text{Conv1d}(k = 2)$
LSTM	长短时记忆网络	128	1	LSTM 单元
Vanilla RNN	简单循环网络	128	1	RNN 单元
ResNet MLP	残差连接	256	2	残差块
Deep MLP	多层 MLP + LayerNorm + Dropout	256	6	Dropout=0.1

所有模型共享相同的嵌入层设计, 输入两个整数的嵌入向量拼接后进行处理。

A.3 训练配置

优化器策略 (子任务 3)

系统比较了 12 种优化器配置, 涵盖不同优化算法与正则化技术:

表 2: Subtask 3 优化器与正则化配置

实验组名称	优化器	学习率	权重衰减	动量	其他参数
AdamW (WD=1.0) [Baseline]	AdamW	10^{-3}	1.0	-	$\beta = (0.9, 0.98)$
Adam (No WD)	Adam	10^{-3}	0.0	-	$\beta = (0.9, 0.999)$
AdamW (WD to Init)	AdamW	10^{-3}	1.0	-	向初始值衰减
AdamW (High WD=3.0)	AdamW	10^{-3}	3.0	-	$\beta = (0.9, 0.98)$
AdamW (Low LR=1e-4)	AdamW	10^{-4}	1.0	-	$\beta = (0.9, 0.98)$
AdamW (Std Betas 0.999)	AdamW	10^{-3}	1.0	-	$\beta = (0.9, 0.999)$
Adam (LR=5e-3)	Adam	5×10^{-3}	0.0	-	$\beta = (0.9, 0.98)$
SGD (Heavy-Ball, mom=0.9)	SGD	0.01	10^{-4}	0.9	Nesterov=False
RMSprop	RMSprop	10^{-3}	10^{-4}	0.0	-
SGD (Nesterov, mom=0.99)	SGD	0.05	10^{-4}	0.99	Nesterov=True
Full-Batch GD	SGD	0.05	0	0.0	批次大小 = 全数据集
AdamW (Weight Noise=0.01)	AdamW	10^{-3}	1.0	-	权重噪声 $\sigma = 0.01$

正则化技术

- **权重衰减 (Weight Decay)**: 主要正则化手段, 显著促进 Grokking
- **Dropout**: 在 DeepMLP 等模型中设为 0.1
- **权重噪声**: 训练时向参数添加高斯噪声, 探索平坦最小值
- **梯度裁剪**: 限制梯度范数 ≤ 1.0 , 防止梯度爆炸

超参数设置

表 3: 实验超参数设置

超参数	取值	说明
批次大小	512	若训练集小于 512 则使用全批次
训练步数	10^5 - 10^6	根据实验调整
学习率预热	前 10 步线性预热	稳定训练初期
早停条件	验证准确率 $> 99.5\%$	提前终止以节省计算
随机种子	{0, 1, 2, 4, 5}	多次实验取中位数

评估指标

- **训练损失**: 交叉熵损失 (CrossEntropyLoss)
- **训练准确率**: 训练集上的预测准确率
- **测试准确率**: 未见数据上的泛化性能
- **Grokking 判定**: 测试准确率从随机水平 ($\sim 1\%$) 突增至接近完美 ($> 99\%$)
- **数据效率曲线**: 达到 99% 测试准确率所需的训练步数随 α 的变化

计算环境

- **硬件**: NVIDIA GPU (本实验结果由 A100 运行得到, 通过 AutoDL 平台租赁)
- **软件**: Python 3.12 + PyTorch 2.9.1 + CUDA 13.0